



GUIDE D'INTÉGRATION

Intégrer EADE dans vos applications et vos outils SIG

Bibliothèque Python, ligne de commande, API REST, OGC API Features, plugin QGIS et fichier projet : comment brancher le moteur de détection adaptatif EADE, l'alimenter, exploiter ses décisions et gouverner son apprentissage.

Version du moteur	Date	Public
0.1.0.dev0	9 octobre 2026	Développeurs, intégrateurs SIG, administrateurs

Sommaire

1. EADE en bref	3
Principes d'intégration	3
2. Choisir son mode d'intégration	4
Arbre de décision rapide	4
3. Installation	5
Accès aux rasters : rasterio ou GDAL	5
4. Concepts et données	6
Données géospatiales en entrée	6
Couche de résultats	6
5. Intégration en Python	7
5.1 Le moteur seul, pour n'importe quel domaine	7
5.2 Détection sur un levé drone	7
5.3 Brancher son propre détecteur	7
5.4 Le cycle complet dans un fichier projet	8
6. Ligne de commande	9
7. API REST	10
Authentification	10
Conventions	10
Routes	11
Exemple de bout en bout	12
8. OGC API Features	13
9. Plugin QGIS	14
Algorithmes de traitement (groupe EADE)	14
Atelier de correction et de validation	14
10. Format des connaissances	15
Grammaire des conditions	15
Import depuis eFoncier (eade-version/1)	15
11. Dictionnaire des caractéristiques (EADE Geo)	16
12. Gouvernance et sécurité	17
Permissions	17
Modes	17
Garanties imposées par la base	17
Exploitation	17
13. Intégrer EADE à eFoncier Africa ou à une plateforme existante	18
Appel depuis un backend Java (Spring)	18
14. Dépannage	19
Limites actuelles et suite	19

1. EADE en bref

EADE (eFoncier Adaptive Detection Engine) est un moteur de décision explicable. Il ne dessine pas : il **juge des candidats** proposés par un fournisseur de détection (le « moteur classique »), mesure chacun d'eux, applique les règles d'une **version de connaissance**, les compare aux cas déjà validés par des experts, et rend une décision accompagnée de son explication complète.

Décision	Condition	Ce qu'en fait l'application hôte
ACCEPTED (retenu)	score $\geq$ seuil d'acceptation (0,60)	Proposer l'objet ; il reste à revoir avant toute promotion métier.
REVIEW (en revue)	entre les deux seuils	Le moteur s'abstient : un opérateur décide. À traiter en premier.
REJECTED (écarté)	score $\leq$ seuil de rejet (0,35)	Masquer par défaut, garder consultable.

Le score est une **force de décision**, pas une probabilité : c'est la moyenne pondérée des composantes disponibles (verdict du moteur classique, règles pondérées, similarité aux cas validés, rampes de caractéristiques comme la hauteur). Trois garde-fous s'appliquent ensuite : une règle bloquante écarte quoi qu'il arrive ; une règle d'abstention renvoie en revue ; un candidat rejeté par le moteur classique ne dépasse jamais la revue.

Principes d'intégration

- **Optionnel.** EADE désactivé, l'application garde son moteur classique tel quel. Aucune écriture hors des données d'EADE : ni barème, ni valeur cadastrale, ni parcelle.
- **Explicable.** Chaque décision porte sa trace JSON : composantes, règles déclenchées avec les valeurs lues, règles non évaluables, garde-fous, version et empreinte de la connaissance.
- **Gouverné.** L'apprentissage ne part que de corrections validées par un expert, produit toujours un brouillon, et une version n'est publiée qu'après évaluation sur un jeu de test indépendant.
- **Interopérable.** GeoTIFF/COG, GeoPackage, GeoJSON, Shapefile via GDAL ; REST/JSON ; OGC API Features.

Statut. ce guide décrit la version de développement du moteur (branche `feat/core-engine` du dépôt). Les interfaces peuvent encore évoluer avant la version 1.0 ; la licence du code sera publiée avant la première version.

2. Choisir son mode d'intégration

Les six formes ci-dessous partagent le même moteur, le même format de connaissance et donnent exactement les mêmes décisions. On les combine librement : un serveur pour la plateforme web, le plugin pour les opérateurs SIG, la bibliothèque pour les traitements de masse.

Forme	Pour qui, pour quoi	Point d'entrée
Bibliothèque Python	Intégrer le moteur dans un traitement, un notebook, un autre logiciel ; brancher son propre détecteur.	<code>import eade</code>
Ligne de commande	Scripts, traitements par lots, chaînes de production.	<code>eade ...</code>
API REST	Plateformes web et applications métier (Java, .NET, PHP, JavaScript...), multi-utilisateurs avec permissions.	<code>eade serve → /api/v1</code>
OGC API Features	Afficher les prédictions dans QGIS, ArcGIS, MapLibre, Leaflet, OpenLayers sans code.	<code>/ogc/collections</code>
Plugin QGIS	Opérateurs et experts SIG : détection, mesure, évaluation, atelier de correction et de validation dans QGIS 3.34+ et 4.x.	<code>eade_qgis-x.y.z.zip</code>
Fichier projet .eade	Un projet complet et portable (SQLite) : campagnes, corrections, exemples, versions, journal.	<code>eade.store.Workspace</code>

Arbre de décision rapide

- Votre application n'est pas en Python, ou plusieurs utilisateurs travaillent en même temps → **API REST**.
- Vous voulez seulement voir les résultats sur une carte → **OGC API Features**.
- Vos opérateurs travaillent dans QGIS → **plugin QGIS** (sur un fichier .eade partagé ou local).
- Vous écrivez un traitement ou votre propre détecteur en Python → **bibliothèque**.

3. Installation

Prérequis : Python 3.10 ou plus récent. Le cœur n'a aucune dépendance ; les capacités s'ajoutent par « extras » pip.

Extra	Apporte	Dépendances
(aucun)	Cœur : règles, décision, versions, apprentissage, évaluation, CLI eade knowledge decide	aucune
geo	Rasters, vecteurs, détection, 37 caractéristiques, projet .eade	numpy, scipy, rasterio, shapely, pyproj, pyogrio
server	API REST et OGC API Features (inclut geo)	fastapi, uvicorn
dev	Tests	pytest, httpx

```
# depuis un clone du dépôt
git clone https://github.com/genidigital/EADE.git && cd EADE
python -m venv .venv && .venv\Scripts\activate          # Windows (source .venv/bin/activate ailleurs)
pip install -e ".[server]"                             # cœur + géospatial + serveur
eade --version

# ou directement depuis GitHub, sans clone
pip install "eade[geo] @ git+https://github.com/genidigital/EADE.git@feat/core-engine"
```

Accès aux rasters : rasterio ou GDAL

eade .geo lit les rasters avec rasterio s'il est installé, sinon avec les liaisons Python de GDAL (osgeo) déjà présentes dans QGIS ou ArcGIS Pro. On n'installe donc jamais une seconde copie de GDAL dans ces logiciels. Pour forcer un choix : variable d'environnement EADE_RASTER_BACKEND=rasterio ou gdal.

Distribution. EADE n'est pas encore publié sur PyPI. Tant que ce n'est pas le cas, installez depuis le dépôt et épinglez un commit précis en production.

4. Concepts et données

Terme	Définition
Candidat	Objet proposé par un fournisseur de détection, avec son verdict propre (<code>classic_accepted</code>) et ses mesures. EADE ne juge que des candidats.
Caractéristique	Mesure déclarée dans un dictionnaire (type, unité, normalisation). Le cœur ignore ce qu'elle signifie.
Règle	Condition JSON + effet : CONFIRM, REJECT (éventuellement bloquante), ABSTAIN, RECLASSIFY. Priorité, poids de 0 à 1, origine EXPERT ou LEARNED.
Versión de connaissance	Réglages de fusion, profils, règles, signatures. Statuts DRAFT → CANDIDATE → PUBLISHED → ARCHIVED. Une version publiée est figée.
Empreinte	SHA-256 de tout ce qui influe sur les décisions. Renommer une règle ne la change pas ; modifier un seuil, si. C'est elle que l'évaluation certifie.
Signature	Portrait moyen (moyennes, écarts) des exemples validés d'une classe, positifs ou négatifs ; sert à la similarité.
Exemple	Correction validée par un expert, avec ses mesures : matière de l'apprentissage et du test.
Maille	Carré UTM de 200 m. Une maille sur cinq est réservée au test, de façon déterministe et définitive.
Profil	Contexte (zone, résolution...) dans lequel des règles et signatures particulières s'appliquent.

Données géospatiales en entrée

- **MNS** (modèle numérique de surface) : obligatoire pour la détection par hauteur ; c'est la grille de référence.
- **MNT** (terrain) : facultatif ; sans lui, le sol est estimé à partir du MNS (ouverture morphologique sur 30 m).
- **Orthophoto** RGB ou RGBA : facultative mais recommandée (couleur, végétation, ombre, texture).
- **Parcelles** : facultatives ; elles ajoutent les caractéristiques de contexte et le rattachement de chaque objet à sa parcelle.
- Projection **métrique** obligatoire pour la grille de référence (UTM par exemple). Les autres rasters peuvent avoir une autre résolution ou projection : ils sont rééchantillonnés à la volée.
- Les rasters sont lus par tuiles de 200 m avec 25 m de recouvrement : une orthophoto géante n'est jamais chargée entière, et un objet à cheval sur deux tuiles sort une seule fois.

Couche de résultats

Champ	Contenu
<code>candidate_id</code>	Identifiant stable du candidat (dérivé de son centroïde)
<code>class / final_class</code>	Classe proposée / classe après reclassement éventuel
<code>decision, score</code>	ACCEPTED, REVIEW ou REJECTED ; score 0–1 (vide si aucune preuve)
<code>classic_accepted, classic_reason</code>	Verdict du moteur classique et motif (SMALL, VEGETATION, LOW)
<code>rules_fired</code>	Codes des règles déclenchées
<code>measure</code>	Surface (m²) ou longueur (m)
<code>parcel_id</code>	Parcelle qui contient la plus grande part de l'objet
<code>knowledge_version, fingerprint</code>	Version et empreinte de la connaissance appliquée
<code>explanation</code>	Trace JSON complète de la décision

5. Intégration en Python

5.1 Le moteur seul, pour n'importe quel domaine

Le cœur ne connaît ni parcelle ni raster. Déclarez vos caractéristiques, écrivez une version et soumettez des candidats :

```
from eade import Candidate, Effect, Engine, FeatureCatalog, FeatureDef, KnowledgeVersion, Rule

catalog = FeatureCatalog([
    FeatureDef("amount", label="Montant", unit="XOF"),
    FeatureDef("signed", dtype="boolean", label="Signé"),
])
version = KnowledgeVersion(number=1, rules=(
    Rule("UNSIGNED", "INVOICE", Effect.ABSTAIN, {"feature": "signed", "op": "=", "value": False}),
))
engine = Engine(version, catalog) # refuse une règle qui cite une caractéristique inconnue
r = engine.decide(Candidate("doc-42", "INVOICE", {"amount": 125000, "signed": False},
    classic_accepted=True))
r.decision          # Decision.REVIEW
r.guardrails        # ('ABSTAIN_RULE:UNSIGNED',)
r.explanation()      # dict JSON : composantes, règles, garde-fous, version, empreinte
```

5.2 Détection sur un levé drone

```
from eade.geo import GeoPipeline, RasterSources
from eade.geo.vector import write_results
from eade.io import knowledge_json

version = knowledge_json.load("connaissance.json") # ou None : verdicts classiques seuls
run = GeoPipeline(version).run(
    RasterSources(dsm="mns.tif", dtm="mnt.tif", ortho="ortho.tif"),
    bounds=None, # (xmin, ymin, xmax, ymax), SCR du MNS
    parcels="parcelles.gpkg", parcel_id_field="PARCELLE",
    progress=lambda done, total: True) # renvoyer False interrompt entre deux tuiles
run.counts() # {'ACCEPTED': ..., 'REVIEW': ..., 'REJECTED': ...}
run.provenance # sources, détecteur, moteur raster, version et empreinte
write_results("predictions.gpkg", run.results, run.crs, extra=["parcel_id"])
```

5.3 Brancher son propre détecteur

Tout fournisseur qui produit des Candidate s'intègre : moteur de la plateforme hôte, réseau de neurones, import d'un autre logiciel. Renvoyez aussi les candidats que votre moteur rejette (classic_accepted=False) : EADE pourra envoyer en revue un rejet douteux. Mesurez-les avec l'extracteur EADE Geo pour bénéficier des règles et signatures existantes :

```
from eade import Candidate, Engine
from eade.geo import GeoFeatureExtractor, RasterSources, geo_catalog

extractor, engine = GeoFeatureExtractor(), Engine(version, geo_catalog())
with RasterSources(dsm="mns.tif", ortho="ortho.tif").open() as src:
    for obj in mon_détecteur(...): # vos géométries shapely, SCR du MNS
        features = extractor.measure(obj.geometry, src)
        result = engine.decide(Candidate(
            obj.id, "BUILDING", features, classic_accepted=obj.ok,
            context={"resolution_m": src.resolution}, geometry=obj.geometry))
```

5.4 Le cycle complet dans un fichier projet

Workspace applique les mêmes règles que le serveur : chaque appel vérifie une permission et s'inscrit au journal.

```
from eade.store import Actor, Workspace

from eade.store import PERMISSIONS

admin = Actor("aminata", frozenset(PERMISSIONS))          # toutes les permissions
with Workspace.create("songon.eade") as ws:
    ws.update_settings(admin, "Pilote Songon", enabled=True, mode="COLLECT")
    c = ws.create_campaign(admin, "Lot 1", {"dsm": "mns.tif", "dtm": "mnt.tif", "ortho": "ortho.tif"},
                             parcels="parcelles.gpkg", parcel_id_field="PARCELLE")
    c = ws.run_campaign(admin, c["id"])                    # tuile par tuile, reprend après interruption
    unit = ws.review_queue(c["id"])[0]["unit"]             # parcelle la plus incertaine
    p = ws.predictions(c["id"], parcel_id=unit)[0]
    ws.correct(admin, "ACCEPT", prediction_id=p["id"])
    ws.submit(admin, c["id"], unit)
    ids = [x["id"] for x in ws.corrections(c["id"], "SUBMITTED")]
    ws.review(admin, ids, approve=True, justification="Conforme à l'orthophoto") # crée les exemples
    candidate, report = ws.build_candidate(admin)          # brouillon appris, rien n'est publié
    ws.simulate(admin, candidate.number, c["id"])           # rejoue sans rien écrire
    ws.evaluate(admin, candidate.number)                   # sur les mailles de test
```


6. Ligne de commande

Commande	Rôle
<code>eade knowledge show FICHER [--catalog C]</code>	Résumé d'une version et de ses règles en clair
<code>eade knowledge import FICHER -o SORTIE</code>	Importer (natif ou eade-version/1) en brouillon
<code>eade knowledge fingerprint FICHER</code>	Afficher l'empreinte
<code>eade decide --knowledge K --catalog C</code>	Candidats JSON Lines en entrée, explications en sortie
<code>eade geo detect --dsm ... -o SORTIE</code>	Détection, mesure et décision sur un levé
<code>eade geo catalog [-o FICHER]</code>	Dictionnaire des caractéristiques en JSON
<code>eade workspace create CHEMIN.eade</code>	Créer un projet vide
<code>eade serve PROJET.eade [--tokens T]</code>	Servir un projet en REST et OGC API Features

```
eade geo detect --dsm mns.tif --dtm mnt.tif --ortho ortho.tif \
    --parcels parcelles.gpkg --parcel-id PARCELLE \
    --knowledge connaissance.json --bounds 368500 590200 369100 590800 \
    -o predictions.gpkg --provenance run.json
# 4 objects -> predictions.gpkg (accepted 2, review 0, rejected 2)

# Moteur seul : une ligne JSON par candidat
echo '{"id": "1", "class": "BATIMENT", "classic_accepted": true,
    "features": {"hauteur_mediane_m": 3.1}}' \
| eade decide --knowledge v2.json --catalog catalog.json
```

Formats de sortie selon l'extension : .gpkg, .geojson, .shp, .fgb. Codes de retour : 0 succès, 2 erreur d'entrée (message préfixé par eade:).

7. API REST

Un serveur sert un projet .eade. Documentation interactive OpenAPI sur /docs, schéma sur /openapi.json (générez un client typé dans votre langage à partir de ce schéma).

```
eade workspace create songon.eade
eade serve songon.eade # local, sans authentication, 127.0.0.1:8765
eade serve songon.eade --host 0.0.0.0 --port 8765 --tokens jetons.json
```

Authentification

Avec --tokens, chaque requête porte Authorization: Bearer <jeton>. Sans fichier de jetons, le serveur refuse d'écouter ailleurs que sur 127.0.0.1. Générez les jetons avec un outil cryptographique (par exemple openssl rand -hex 32) et ne versionnez jamais ce fichier.

```
{
  "9f2c...e1": {"user": "opérateur.songon", "permissions": ["VIEW", "RUN", "REVIEW"]},
  "41ab...7d": {"user": "expert.cadastre", "permissions": ["VIEW", "VALIDATE"]},
  "c03e...90": {"user": "responsable.eade", "permissions": ["VIEW", "QUALITY", "PUBLISH", "ADMIN"]}
}
```

Conventions

- Géométries en **GeoJSON WGS 84** (RFC 7946) par défaut ; crs=native (paramètre de requête ou champ crs du corps) pour travailler dans la projection de la campagne.
- Un contour envoyé sans features est **mesuré par le serveur** sur les rasters de la campagne (REDRAW, ADD, SPLIT, MERGE).
- Les opérations longues répondent **202** ; on suit la campagne par GET /campaigns/{id} (status PENDING, RUNNING, DONE, FAILED, CANCELLED, INTERRUPTED).
- Erreurs : corps {"error": "..."} ; **401** jeton absent ou inconnu, **403** permission manquante, **404** introuvable, **409** conflit d'état (version figée, campagne déjà en cours, évaluation absente), **422** donnée invalide (justification, motif, géométrie), **503** rasters inaccessibles.

Routes

Méthodes	Chemin (préfixe /api/v1)	Permission	Rôle
GET	/health	—	État du service, mode EADE, version active
GET	/me	VIEW	Utilisateur et permissions du jeton
GET	/catalog	VIEW	Dictionnaire des caractéristiques
GET	/settings	VIEW / ADMIN	Lire ; PATCH : activer, mode, critères (justification)
PATCH	/settings	VIEW / ADMIN	Lire ; PATCH : activer, mode, critères (justification)
POST	/settings/emergency-stop	ADMIN	Désactiver EADE et annuler les campagnes qui l'appliquent
GET	/versions	VIEW / ADMIN	Lister ; POST : nouveau brouillon
GET	/versions/{number}	VIEW / ADMIN	Exporter ; PUT : enregistrer un brouillon ; DELETE
POST	/versions	VIEW / ADMIN	Lister ; POST : nouveau brouillon
PUT	/versions/{number}	VIEW / ADMIN	Exporter ; PUT : enregistrer un brouillon ; DELETE
POST	/versions/import	ADMIN	Importer un fichier comme brouillon
DELETE	/versions/{number}	VIEW / ADMIN	Exporter ; PUT : enregistrer un brouillon ; DELETE
POST	/versions/{number}/publish	PUBLISH	Publier (évaluée ou manuelle)
POST	/versions/{number}/rollback	PUBLISH	Réactiver une version publiée
POST	/versions/{number}/simulate	QUALITY	Rejouer sur une campagne, sans écriture
POST	/versions/{number}/evaluate	QUALITY	Évaluer sur le jeu de test
GET	/versions/{number}/evaluations	VIEW	Historique des évaluations
GET	/learning	VIEW	Exemples par classe, polarité, jeu
POST	/learning/candidate	ADMIN	Construire une version candidate
GET	/campaigns	VIEW / RUN	Lister ; POST : créer et lancer (202)
POST	/campaigns	VIEW / RUN	Lister ; POST : créer et lancer (202)
GET	/campaigns/{campaign_id}	VIEW	État, décompte, provenance
POST	/campaigns/{campaign_id}/resume	RUN	Reprendre une campagne interrompue
POST	/campaigns/{campaign_id}/cancel	RUN	Arrêter entre deux tuiles
GET	/campaigns/{campaign_id}/predictions	VIEW	GeoJSON ; filtres decision, parcel_id, bbox, crs
GET	/predictions/{prediction_id}	VIEW	Une prédiction avec mesures et explication
GET	/campaigns/{campaign_id}/queue	VIEW	File de revue : uncertainty, random, order
GET	/campaigns/{campaign_id}/corrections	VIEW	Corrections, filtre status
POST	/corrections	REVIEW	ACCEPT, REJECT, REDRAW, RECLASSIFY, ADD
POST	/corrections/split	REVIEW	Scinder un objet
POST	/corrections/merge	REVIEW	Fusionner des objets
DELETE	/corrections/{correction_id}	REVIEW	Retirer un brouillon

Méthodes	Chemin (préfixe /api/v1)	Permission	Rôle
POST	/campaigns/{campaign_id}/submit	REVIEW	Soumettre ses brouillons à l'expert
POST	/corrections/review	VALIDATE	Valider ou rejeter (justification)
GET	/audit	VIEW	Journal inaltérable

Exemple de bout en bout

API=http://serveur:8765/api/v1 ; H="Authorization: Bearer \$JETON" ; J="Content-Type: application/json"

```
# 1. lancer une campagne (chemins vus par le serveur)
curl -s -X POST $API/campaigns -H "$H" -H "$J" -d '{
  "label": "Section HT", "dsm": "/data/songon/mns.tif", "dtm": "/data/songon/mnt.tif",
  "ortho": "/data/songon/ortho.tif", "parcels": "/data/songon/parcelles.gpkg",
  "parcel_id_field": "PARCELLE"}'
```

2. suivre, puis récupérer les objets en revue (GeoJSON WGS 84)

```
curl -s $API/campaigns/1 -H "$H"
curl -s "$API/campaigns/1/predictions?decision=REVIEW&limit=500" -H "$H"
```

3. corriger : redessiner un bâtiment (mesuré par le serveur), puis soumettre la parcelle

```
curl -s -X POST $API/corrections -H "$H" -H "$J" -d '{
  "action": "REDRAW", "prediction_id": 1542,
  "geometry": {"type": "Polygon", "coordinates": [[[-4.2241, 5.3402], ...]]}}'
```

```
curl -s -X POST $API/campaigns/1/submit -H "$H" -H "$J" -d '{"parcel_id": "HT/112/1091"}'
```

4. l'expert valide

```
curl -s -X POST $API/corrections/review -H "Authorization: Bearer $JETON_EXPERT" \
  -H "$J" -d '{"ids": [87, 88], "approve": true, "justification": "Vérifié"}'
```

5. apprendre, simuler, évaluer, publier

```
curl -s -X POST $API/learning/candidate -H "$H" -H "$J" -d '{}'
```

```
curl -s -X POST $API/versions/3/simulate -H "$H" -H "$J" -d '{"campaign_id": 1}'
```

```
curl -s -X POST $API/versions/3/evaluate -H "$H"
```

```
curl -s -X POST $API/versions/3/publish -H "$H" -H "$J" \
  -d '{"justification": "Gain IoU +0,04 sur 120 objets de test"}'
```

8. OGC API Features

Chaque campagne est publiée comme une collection `campaign-{id}` conforme à OGC API Features Part 1 (Core, GeoJSON). Les mêmes jetons s'appliquent.

URL	Contenu
/ogc	Page d'accueil (liens conformance, collections)
/ogc/conformance	Classes de conformité
/ogc/collections	Une collection par campagne lancée
/ogc/collections/campaign-1/items?bbox=...&limit=...&offset=...	Prédictions en GeoJSON WGS 84

- **QGIS** : Couche > Ajouter une couche > WFS / OGC API Features, URL `http://serveur:8765/ogc`, version « OGC API - Features ».
- **Cartes web** : chargez `.../items` comme source GeoJSON et stylez sur `decision` et `status`.

```
// MapLibre GL : objets d'une campagne, colorés par décision
map.addSource("eade", { type: "geojson",
  data: "http://serveur:8765/ogc/collections/campaign-1/items?limit=5000" });
map.addLayer({ id: "eade", type: "line", source: "eade", paint: {
  "line-width": 2,
  "line-color": ["match", ["get", "decision"],
    "ACCEPTED", "#f07c1e", "REVIEW", "#e69600", "REJECTED", "#788291", "#000"] } });
```

Attention. un navigateur ne peut pas ajouter d'en-tête `Authorization` à une source de tuiles ou de GeoJSON chargée directement par la bibliothèque cartographique. Pour une carte publique, servez les résultats par un proxy de votre application qui ajoute le jeton côté serveur.

9. Plugin QGIS

- Compatible QGIS 3.34 et plus, et QGIS 4.x (Qt5 et Qt6). Le moteur est embarqué : rien à installer, il utilise le GDAL de QGIS.
- Construction : `python scripts/build_qgis_plugin.py` produit `dist/eade_qgis-x.y.z.zip` ; installation : Extensions > Installer depuis un ZIP.
- Données d'essai : `python scripts/make_demo.py` écrit un levé de 40 m × 40 m, des parcelles et un projet vide dans `dist/demo`.

Algorithmes de traitement (groupe EADE)

Identifiant	Rôle	Sorties
eade:detect	Détecter, mesurer et décider sur un levé (parcelles, connaissance et emprise facultatives)	couche, ACCEPTED, REVIEW, REJECTED
eade:measure	Mesurer des emprises avec le dictionnaire EADE Geo	couche
eade:evaluate	Comparer des détections à des références : IoU, précision, rappel	MEAN_IU, PRECISION, RECALL, FALSE_POSITIVES, MISSES, REPORT
eade:campaign	Enregistrer et exécuter une campagne dans un projet .eade	couche, CAMPAIGN

Utilisables dans le modeleur graphique, en lot, ou en Python depuis la console QGIS :

```
import processing
res = processing.run("eade:detect", {
    "DSM": "D:/songon/mns.tif", "DTM": "D:/songon/mnt.tif", "ORTHO": "D:/songon/ortho.tif",
    "PARCELS": "D:/songon/parcelles.gpkg", "PARCEL_ID": "PARCELLE",
    "KNOWLEDGE": "D:/songon/v3.json", "OUTPUT": "memory:{}".format("}")
})
QgsProject.instance().addMapLayer(res["OUTPUT"])
```

Atelier de correction et de validation

Panneau ancré à côté de la carte, ouvert par l'icône EADE. Il charge les objets d'une campagne (orange retenu, jaune en revue, gris pointillé écarté, vert corrigé, rouge retiré), propose la file des parcelles et la fiche explicative de chaque objet. Raccourcis : **A** accepter, **R** rejeter, **1-8** motif, **D** redessiner, **B** bâtiment manquant, **N/P** parcelle suivante/précédente, **Ctrl+Entrée** soumettre. Les contours se tracent avec les outils de numérisation de QGIS sur la couche « EADE – tracés » et sont mesurés automatiquement.

Limite actuelle. en mode fichier, l'atelier agit sous le nom de l'utilisateur Windows avec toutes les permissions : la séparation opérateur / expert n'y est pas imposée. Pour une équipe, centralisez le projet derrière eade serve avec des jetons par rôle.

10. Format des connaissances

Une version s'échange en JSON `eade.knowledge/1`. L'empreinte est vérifiée à l'import : un fichier modifié après export est refusé. Un import devient toujours un brouillon, à évaluer sur le jeu de test de l'installation qui le reçoit.

```
{
  "format": "eade.knowledge/1", "number": 3, "label": "Règles expertes", "status": "DRAFT",
  "fingerprint": "4aa04cfa3de7...",
  "config": {
    "weights": {"classic": 1, "rules": 1, "similarity": 1},
    "thresholds": {"accept": 0.6, "reject": 0.35},
    "ramps": [{"name": "height", "feature": "height_median_m", "low": 2, "full": 2.6,
      "weight": 0.5, "classes": ["BUILDING"]}],
    "similarity": {"min_count": 5, "max_distance": 3}
  },
  "profiles": [],
  "rules": [{
    "code": "SHADOW", "label": "Emprise surtout dans l'ombre", "target_class": "BUILDING",
    "effect": "ABSTAIN", "condition": {"feature": "shadow_share", "op": ">=", "value": 0.5},
    "weight": 0.5, "priority": 70, "blocking": false, "active": true, "origin": "EXPERT"},
    "signatures": []
  ]
}
```

Grammaire des conditions

Forme	Sens
<code>{"all": [c1, c2, ...]}</code>	toutes vraies (ET)
<code>{"any": [c1, c2, ...]}</code>	au moins une vraie (OU)
<code>{"not": c}</code>	négation
<code>{"feature": "k", "op": "<", "value": 3}</code>	comparaison : <code>< <= > >= !=</code>
<code>{"feature": "k", "between": [a, b]}</code>	$a \leq k \leq b$
<code>{"feature": "k", "in": [v1, v2]}</code>	appartenance

Profondeur maximale 6, 50 nœuds au plus. Évaluation à trois valeurs : une caractéristique absente rend la condition **inconnue** ; une règle ne s'applique que si sa condition est vraie, et les règles non évaluables sont signalées dans l'explication.

Import depuis eFoncier (eade-version/1)

Attention. les règles importées gardent les noms de caractéristiques d'eFoncier (par exemple `hauteur_mediane_m`). Pour les appliquer avec le détecteur EADE Geo, renommez-les vers le dictionnaire de la section 11 (`height_median_m...`), sinon Engine refusera la version. L'empreinte EADE diffère de celle d'eFoncier : réévaluez avant de publier.

eFoncier	EADE
effet CONFIRMER / ECARTER / ABSTENIR / RECLASSER	CONFIRM / REJECT / ABSTAIN / RECLASSIFY
poids classique / règles / cas	weights classic / rules / similarity
poids hauteur + hauteur.{classe}.min / plein	rampe height sur hauteur_mediane_m
seuils retenu / ecarte	thresholds accept / reject
similarite effectif_min / distance_max	similarity min_count / max_distance
et / ou / non, car, valeur, entre, dans	all / any / not, feature, value, between, in
origine APPRENTISSAGE ; profil DEFAULT	origin LEARNED ; règle générale

11. Dictionnaire des caractéristiques (EADE Geo)

37 caractéristiques, version d'extracteur geo-1. Les contextuelles ne sont calculées que si des parcelles sont fournies. « Non appris » : la caractéristique décrit l'acquisition et n'est jamais utilisée pour proposer un seuil.

Clé	Groupe	Libellé	Unité	Normalisation
area_m2	géométrie	Surface	m ²	log1p ÷ 8
perimeter_m	géométrie	Périmètre	m	log1p ÷ 6
compactness	géométrie	Compacité	—	—
rectangularity	géométrie	Rectangularité	—	—
elongation	géométrie	Allongement	ratio	log1p ÷ 3
orientation_deg	géométrie	Orientation	deg	÷ 180 · non appris
width_m	géométrie	Largeur	m	log1p ÷ 4
vertex_count	géométrie	Nombre de sommets	—	log1p ÷ 5
height_median_m	altimétrie	Hauteur médiane au-dessus du sol	m	÷ 10
height_p90_m	altimétrie	Hauteur P90	m	÷ 10
height_p10_m	altimétrie	Hauteur P10	m	÷ 10
height_spread_m	altimétrie	Écart de hauteur P90 – P25	m	÷ 5
slope_mean	altimétrie	Pente moyenne de la surface	m/m	÷ 2
steep_edge_share	altimétrie	Part du pourtour en mur (pente > 45°)	—	—
curvature_median	altimétrie	Courbure médiane de la surface	1/m	÷ 0.5
rough_share	altimétrie	Part de surface rugueuse	—	—
red_mean	spectral	Rouge moyen	0-255	÷ 255
green_mean	spectral	Vert moyen	0-255	÷ 255
blue_mean	spectral	Bleu moyen	0-255	÷ 255
brightness	spectral	Luminosité	0-255	÷ 255
contrast	spectral	Contraste	0-255	÷ 64
saturation	spectral	Saturation	—	—
hue_cos	spectral	Teinte (cosinus)	—	—
hue_sin	spectral	Teinte (sinus)	—	—
excess_green	spectral	Indice d'excès de vert	—	—
green_share	spectral	Part de pixels verts	—	—
local_variance	texture	Variance locale	—	log1p ÷ 6
gradient_mean	texture	Gradient moyen	0-255	÷ 64
edge_density	texture	Densité de contours	—	—
elevation_coverage	qualité	Couverture du MNS	—	—
shadow_share	qualité	Part dans l'ombre	—	—
resolution_m	qualité	Taille du pixel	m	— · non appris
truncated	qualité	Coupé par le bord des données	—	booléen
parcel_share	contexte	Part dans sa parcelle	—	—
parcel_count	contexte	Parcelles touchées	—	÷ 5
boundary_distance_m	contexte	Distance du centre à la limite	m	log1p ÷ 3
touches_boundary	contexte	Touche la limite de parcelle	—	booléen

12. Gouvernance et sécurité

Permissions

Permission	Pour qui	Permet
VIEW	Tous	Consulter campagnes, prédictions, connaissances, journal
RUN	Opérateur, chef d'équipe	Lancer, arrêter, reprendre une campagne
REVIEW	Opérateur	Corriger et soumettre
VALIDATE	Expert	Valider ou rejeter les corrections soumises
QUALITY	Qualité	Simuler et évaluer une version
PUBLISH	Responsable EADE	Publier, rétablir une version
ADMIN	Responsable EADE	Activer, mode, règles, candidates, import, arrêt d'urgence

Modes

Mode	Campagnes	Corrections validées	Publication
Désactivé	Moteur classique seul	Enregistrées, sans apprentissage	—
READ_ONLY	Classique + version active	Enregistrées, sans apprentissage	Manuelle
COLLECT	Classique + version active	Deviennent des exemples	Manuelle
ADMIN	Classique + version active	Deviennent des exemples	Évaluée ou manuelle

Conditions de publication évaluée : EADE actif en mode ADMIN ; évaluation acceptée de cette empreinte exacte contre la version active ; gain d'IoU moyen $\geq 0,02$ sur au moins 20 objets de test ; ni l'erreur de surface ni les faux positifs n'augmentent ; justification. La publication manuelle exige 15 caractères de justification et garde l'état de la dernière évaluation au journal.

Garanties imposées par la base

- Une prédiction ne change plus, sauf son statut de revue ; elle ne se supprime pas.
- Une version publiée est figée ; un exemple validé est immuable ; une correction jugée ne change plus.
- Le journal est en ajout seul : ni modification ni suppression, même en ouvrant le fichier avec un autre outil.
- Chaque prédiction garde la version et l'empreinte qui l'ont produite : un retour arrière dit exactement quels objets revoir.

Exploitation

- **Arrêt d'urgence** : POST /settings/emergency-stop désactive EADE immédiatement et annule les campagnes qui l'appliquent ; le moteur classique continue.
- **Sauvegarde** : le projet est un seul fichier SQLite ; copiez-le serveur arrêté, ou avec sqlite3 projet.eade ".backup copie.eade". Les rasters se sauvegardent à part.
- **Réseau** : sans jetons, écoute locale uniquement ; en production, placez le serveur derrière un proxy HTTPS et limitez l'accès aux chemins de rasters au serveur lui-même.

13. Intégrer EADE à eFoncier Africa ou à une plateforme existante

Le schéma recommandé garde le moteur classique de la plateforme comme fournisseur et place EADE derrière une interface optionnelle :

- **Étape 1 – Observer.** Servez un projet EADE et lancez des campagnes en mode READ_ONLY sur des zones déjà expertisées. Affichez les prédictions via OGC API Features, sans rien changer au flux métier.
- **Étape 2 – Reprendre la connaissance.** Importez les versions existantes (POST /versions/import), renommez les caractéristiques vers le dictionnaire EADE Geo, simulez sur une campagne de référence.
- **Étape 3 – Collecter.** Passez en mode COLLECT ; les corrections validées par les experts deviennent des exemples. Utilisez régulièrement la file aléatoire pour mesurer honnêtement la qualité.
- **Étape 4 – Améliorer.** Construisez des candidates, évaluez-les, publiez en mode ADMIN seulement quand le gain est prouvé. Rétablissez la version précédente au moindre doute.
- **Toujours :** les valeurs cadastrales et les barèmes restent calculés par la plateforme, sur des objets validés. EADE n'écrit que dans son propre projet.

Appel depuis un backend Java (Spring)

```
RestClient eade = RestClient.builder()
    .baseUrl("http://eade:8765/api/v1")
    .defaultHeader(HttpHeaders.AUTHORIZATION, "Bearer " + jetonService)
    .build();

Map<?, ?> campagne = eade.post().uri("/campaigns")
    .contentType(MediaType.APPLICATION_JSON)
    .body(Map.of("label", "Section HT", "dsm", cheminMns, "dtm", cheminMnt, "ortho", cheminOrtho,
        "parcels", cheminParcelles, "parcel_id_field", "PARCELLE"))
    .retrieve().body(Map.class); // 202 : la campagne tourne en arrière-plan

String geojson = eade.get()
    .uri("/campaigns/{id}/predictions?decision=REVIEW&crs=native", campagne.get("id"))
    .retrieve().body(String.class);
```

Dégradation contrôlée. si EADE est indisponible (erreur réseau, 503), la plateforme doit continuer avec son moteur classique, exactement comme quand EADE est désactivé.

14. Dépannage

Message ou symptôme	Cause et solution
<code>... is not projected: reproject the data to a metric CRS</code>	Le MNS (ou l'orthophoto de référence) est en degrés. Reprojectez-le en UTM ou dans une projection métrique locale.
<code>the height detector needs a surface model (DSM)</code>	La détection par hauteur exige un MNS. Sans MNS, utilisez votre propre fournisseur de candidats (section 5.3).
<code>invalid rules in version N: ... unknown feature</code>	Une règle cite une caractéristique absente du dictionnaire utilisé : corrigez le nom (souvent après un import eFoncier).
<code>fingerprint mismatch</code>	Le fichier de connaissance a été modifié après export. Réexportez-le.
<code>409 a campaign is already running</code>	Une seule campagne à la fois par projet. Attendez, ou annulez-la (POST /campaigns/{id}/cancel).
<code>409 no accepted evaluation of this exact version</code>	Évaluez la version dans son état actuel ; toute modification change l'empreinte.
Campagne INTERRUPTED	Le serveur s'est arrêté pendant le calcul : POST /campaigns/{id}/resume reprend à la tuile suivante, sans doublon.
Peu ou pas d'objets détectés	Vérifiez que MNS et MNT ont la même référence altimétrique ; sans MNT, le sol est estimé sur 30 m et peut sous-estimer de grands bâtiments.
Le plugin n'apparaît pas dans QGIS	Activez-le dans Extensions > Installées ; en version expérimentale, cochez « Afficher les extensions expérimentales ».

Limites actuelles et suite

- Les seuils du détecteur par hauteur sont réglés sur données de synthèse : à calibrer sur un vrai levé avant usage en production.
- L'atelier QGIS travaille sur un fichier ; sa connexion au serveur multi-utilisateurs est à venir.
- Le logiciel de bureau EADE Studio et son installateur Windows sont en cours de développement.
- Publication sur PyPI et choix de la licence avant la version 1.0.